Machine Learning

The Problem Brief — What the Task Is and What the Team Tried to Solve

Read this before you open the code. It describes the business situation, the task the data team was given, the data they had to work with, and what they built. It deliberately does not tell you whether their solution is any good — that is your job.

1. The business situation

A retail bank issues personal loans of between roughly 8,000 and 300,000 shekels. Today, every application is reviewed by a human underwriter against a set of written rules that have barely changed in a decade: minimum credit score, maximum debt-to-income ratio, minimum employment history.

Two things are wrong with this from the business's point of view:

- It is slow and expensive. Each application costs staff time, and competitors approve online in minutes.
- It is crude. The rules use a handful of thresholds, so the bank suspects it is rejecting customers who would have repaid, while approving some who will not.

Around 14% of the loans the bank issues end in default — the borrower stops paying and the bank writes off part or all of the principal. Every default is expensive; every unnecessary rejection is a lost customer and a lost interest margin. The two errors do not cost the same, and that asymmetry matters later.

2. The task the data team was given

> *"We receive thousands of new loan applications every month. Build a model that tells us which applications to approve and which to reject."*

Note what this request implies, because everything else follows from it:

- The decision is made about a NEW applicant — a person who does not yet have a loan with us.
- It is made on ONE specific day: the day the application is submitted.
- It is a yes/no decision: approve or reject.
- Whatever the model uses to decide must therefore be something the bank knows about that person on that day.

There is one more constraint, from outside the business. The regulator requires the bank to be able to explain, in plain terms, why any particular application was rejected. A model the bank cannot explain is a model the bank cannot fully use.

3. The data the team had

The team exported a table from the bank's data warehouse. The warehouse tracks the loans the bank currently has on its books, and the billing system writes a record for each active loan every month. The export therefore contains a monthly history of existing loans.

The file you receive, loans.csv, has 6,557 rows and nine columns:

Column	What it holds
customer_id	An identifier for the borrower
month	Which month of the loan this record describes
income	The borrower's reported income

self_employed	Whether the borrower is self-employed (1) or salaried (0)
credit_score	The credit-bureau score, on a scale of 300–850
loan_amount	The size of the loan, in shekels
avg_days_late	How late the borrower's payments have run, in days
collections_flag	Whether the collections department has been involved with this borrower
default	The outcome: 1 if this loan ended in default, 0 otherwise

Some values are missing: a number of borrowers have no recorded income, and some have no credit score. This is normal in banking data and the pipeline has to do something about it.

4. What the team built

Working quickly with an AI coding assistant, the team produced loan_pipeline.ipynb — a short, entirely standard machine-learning pipeline. It is the shape of pipeline you would find in any textbook or tutorial, and it runs from top to bottom without a single error. In six steps:

1. Load the data from the warehouse and save it as loans.csv, plus a histogram of the income column.

2. Clean it: fill in the missing values so that no gaps remain, and put all the columns on a comparable scale.

3. Split the data into a training portion and a testing portion, so that performance can be measured on data the models have not seen.

4. Train the first model — a Random Forest, an ensemble of decision trees.

5. Train the second model — a neural network with three hidden layers.

6. Compare the two models' accuracy on the test portion, draw a comparison chart, and recommend one of them.

What the pipeline reports

Both models come out at roughly 98% accuracy on the test portion. The notebook's closing lines read:

> *"Both models are around 98% — far better than a 50/50 coin flip. Decision: deploy the NEURAL NETWORK (deep learning is the more advanced technology)."*

On the strength of that, the team has asked for approval to move the model into production, where it would begin making real approval decisions about real applicants.

5. What is actually being asked of you

You own the lending product, and the go/no-go decision is yours. You are not the engineer: nobody expects you to write this pipeline, and you will not be asked to. What you are expected to do is what a business owner has to do whenever a technical team brings a recommendation:

- Understand what the pipeline actually does, step by step, in business terms.
- Work out whether the problem it solves is the problem you asked about.
- Decide which concerns about it are real and which are noise.
- Decide what should be done about the real ones.
- Choose between the two models — or decide that neither is ready — and be able to defend that choice.

> *The pipeline runs cleanly and reports an impressive number. Neither of those facts is evidence that it works. Working out the difference between 'this code ran' and 'this model will make the bank money' is the whole point of this exercise.*

A note on how this code was written

The pipeline was produced quickly, with the help of an AI coding assistant. This is now how a great deal of professional code is written, including in banks, and it is not in itself a reason for suspicion: the assistant produced clean, conventional, readable code.

It does, however, change what your job is. When code is generated in minutes rather than weeks, the bottleneck moves from writing to judging. Somebody has to ask whether the thing that was built

answers the question that was asked — and that somebody is increasingly the person who owns the business problem, not the person who owns the keyboard.

So treat every claim in the notebook, and every claim in this brief, the same way: as something to be checked, not something to be believed.

Preparation Questions

Fifteen short questions to work through at home before the exam. They cover everything the exam covers; if you can answer all fifteen clearly, you are prepared.

How to use them:

Run loan_pipeline.ipynb first — several questions refer to it directly, and the answers are much easier to see once you have run the code and looked at loans.csv and the two charts. Then write a short answer to each question, a paragraph or so is enough.

You may use an AI assistant to help you work through them, on one condition: read the answer critically rather than copying it. The exam asks you to judge work that was produced with an AI assistant and looks convincing — so an answer you accepted without checking is exactly the habit the exam is designed to catch. Where a question refers to our pipeline, verify the answer against the code and the data yourself.

Bring anything you could not resolve to the Zoom session.

---

1. Overfitting and underfitting

What is overfitting, and what is underfitting? For each one, describe what you would see when you compare performance on the training data with performance on the test data, and give one way to fix it.

2. Bias

What does it mean to say a model has high bias? How does this relate to how complex the model is, and how does it relate to underfitting?

3. Accuracy and baselines

About 14% of the loans in our data end in default. What accuracy would you get by simply predicting "no default" for every customer? Given that, why is accuracy a weak way to judge this model, and which measures would show you how it actually handles defaulters?

4. Regression vs classification

What is the difference between regression and classification? Give one banking example of each, and say which of the two the loan-approval problem is.

5. Deep learning

What is deep learning, and when is a neural network the better choice? For a table of about 6,500 rows and 7 columns, would you expect a neural network or a tree-based model to do better, and why?

6. Ensembles

What is an ensemble of models? Explain simply why combining many decision trees usually works better than a single tree, and what makes the individual trees in a Random Forest different from one another.

7. **Parameter vs hyperparameter**

What is the difference between a parameter and a hyperparameter? Give two examples of each from loan_pipeline.ipynb, and say where hyperparameters should be chosen.

8. **Train, validation and test**

What is each of the three sets — training, validation and test — for? What goes wrong if you choose your settings by testing them on the test set, and what goes wrong if there is no validation set at all, as in our pipeline?

9. **Data leakage**

What is data leakage? Give the general definition, then name the columns in loans.csv that leak and explain exactly why each one cannot be used to decide on a new applicant.

10. **What a row represents**

What does one row of loans.csv represent, and how many rows does a single customer have? Explain why splitting the data randomly by row is a problem here, and what kind of split solves it.

11. **Missing values**

In the pipeline, missing income is filled with the mean of the whole table and missing credit_score is filled with 0. Give three separate problems with those two lines, and say what should be done instead.

12. **Order of operations**

Why is it a mistake to fit a StandardScaler, or to compute an imputation value, before splitting the data? What is the correct order, and what is this kind of mistake called?

13. **Reading the data**

The income histogram produced by the pipeline shows two separate peaks, one about twelve times the other. What is the most likely cause, and what should be done about it before any modelling continues?

14. **How the data was collected**

Our training data contains only loans that the bank approved in the past. What is this problem called, why does it matter when the model goes into production, and can it be fixed by changing the code?

15. **Choosing between two models**

The pipeline reports about 99% for the Random Forest and 98% for the neural network, and then recommends the neural network because "deep learning is the more advanced technology". List the criteria you would actually use to choose between two models, and explain why the comparison chart — whose y-axis runs from 0.90 to 1.00 — is misleading.